Informe — TP8 Food Store

1.a Datos de la tablet

Viajan en la instantánea	No viajan
ID, nombre, precio entero en centavos, stock observado, disponibilidad, baja lógica y fecha de copia	Clientes, correos, teléfonos, pedidos confirmados, credenciales y datos de pago

La tablet necesita los datos mínimos para consultar el catálogo y preparar borradores sin conexión. PostgreSQL es la fuente de verdad de precio, stock, vigencia y ventas; SQLite conserva una copia fechada.

2.a–2.b Instantánea y semáforo

La vista `v_snapshot_movil` exporta productos sin clientes ni pedidos y conserva productos inactivos como lápidas. El semáforo procesa únicamente filas `PENDIENTE` y solo actualiza SQLite:

Prioridad	Condición	Estado
Roja	Producto inexistente, no disponible, dado de baja o stock insuficiente	`RECHAZADO`
Amarilla	No hay condición roja, pero cambió el precio visto	`CONFLICTO`
Verde	Los datos coinciden con la instantánea	`SIMULADO_APROBADO`

`SIMULADO_APROBADO` no confirma ni registra una venta.

2.c Casos comprobados

Los cambios de precio y baja se hicieron intencionalmente en la copia PostgreSQL de pruebas; el semáforo funcionó localmente sobre las instantáneas importadas.

Caso	Preparación y resultado observado
C2	Se subió el precio del producto 1 de 1000 a 1100. La solicitud original quedó `CONFLICTO`; al actualizar el precio visto y crear otra solicitud con UUID nuevo, quedó `SIMULADO_APROBADO`.
C3	Borrador por 11 unidades con stock 10: `RECHAZADO`.
C4	La solicitud `...0204` se había evaluado antes de la baja y conserva su resultado histórico `SIMULADO_APROBADO`. Después se marcó el producto 1 inactivo, se exportó/importó una instantánea nueva y la solicitud `...0205` quedó `RECHAZADO`. La fila del producto se conserva como lápida (`disponible=f`, `eliminado=t`) y se oculta del catálogo local disponible.
C5	Borrador sin cambios: `SIMULADO_APROBADO`.
C6	La solicitud `...0206` quedó `SIMULADO_APROBADO`. Reenviar el mismo UUID produjo el error de clave primaria; se comprobó que queda una sola fila. La base conserva el borrador, el ítem y la solicitud relacionados.

En la copia PostgreSQL, la consulta final mostró **5 pedidos** y **stock total 25**, iguales a los valores iniciales. Los únicos cambios de servidor fueron el precio de producto 1 para C2 y la baja lógica de producto 1 para C4; el móvil no creó pedidos ni cambió stock.

3.a Mapa de los tres motores

Motor	Qué guarda	Quién manda
PostgreSQL	Categorías, productos, clientes, pedidos y líneas	Fuente de verdad para catálogo, precio, stock y ventas
MongoDB	Colección documental de pedidos con cliente y líneas embebidos, total y precio histórico	Vista desnormalizada para consulta; PostgreSQL mantiene la autoridad transaccional
SQLite	Instantánea de productos, borradores, ítems y cola local	Administra borradores; no cambia el catálogo ni confirma ventas

Los documentos MongoDB usan datos sintéticos. SQLite no recibe datos de clientes.

3.b Mantenimiento

Respaldar PostgreSQL con `pg_dump` y conservar los scripts versionados. Respaldar SQLite si hay borradores que el equipo necesita recuperar. Probar restauraciones periódicamente en una base nueva y comprobar conteos, restricciones y consultas clave. Una persona responsable revisa conflictos y rechazos al cierre de cada jornada y antes de reintentar.

Si PostgreSQL se restaura a un punto anterior, cada tablet descarga una instantánea nueva y vuelve a comparar los borradores pendientes. Los estados locales anteriores son historial de simulación, no ventas.

3.c Revisión de código asistida por IA

Se revisaron manualmente los scripts SQL de TP8 con Codex: `movil/schema_sqlite.sql`, `movil/snapshot_sqlite.sql`, `movil/semaforo.sql` y `servidor/snapshot_postgresql.sql`. Las comprobaciones indicadas se ejecutaron en una SQLite temporal en memoria; no se modificaron esos scripts. Además, una sesión posterior de OpenCode produjo la revisión estática resumida más abajo para `food store`, `TP2_Food_Store` y `tp8`; según el resultado compartido, esa sesión no modificó ni ejecutó SQL. Las dos revisiones se mantienen separadas para distinguir lo comprobado de lo pendiente.

Hallazgo	Decisión	Cómo se comprobó
`schema_sqlite.sql` permite guardar un borrador sin ítems; `semaforo.sql` puede marcar como aprobado simulado una solicitud pendiente vacía porque no encuentra ítems con errores.	Aceptado como defecto a corregir antes de usar en una app. Recomendación: impedir solicitudes vacías antes de encolarlas o rechazarlas en el semáforo. No se cambió el script.	En SQLite temporal se insertó borrador válido y solicitud pendiente sin filas en `item_borrador`; al ejecutar `semaforo.sql` el estado resultó `SIMULADO_APROBADO`.
`snapshot_sqlite.sql` solo acepta `t`/`f`; un CSV de DBeaver que represente booleanos como `true`/`false` no se puede importar directamente.	Aceptado como incompatibilidad de importación con DBeaver. El CSV exportado con `psql` usa `t`/`f`; normalizar si se usa DBeaver.	En SQLite temporal, insertar `true` falló por el CHECK `disponible IN ('t','f')`; insertar `t` fue aceptado.
En `snapshot_postgresql.sql`, una categoría inactiva da `disponible=false` aunque el producto no esté dado de baja (`eliminado=false`).	Descartado marcarlo como eliminado: la baja de categoría no es una baja lógica del producto. `disponible=false` ya hace que el semáforo rechace borradores mientras esa categoría siga inactiva.	Un fixture SQLite con `disponible='f'`, `eliminado='f'` dio `RECHAZADO` al ejecutar `semaforo.sql`. La expresión de la vista confirma que disponibilidad considera `categoria.activo`, mientras `eliminado` representa `producto.activo`.

Revisión estática adicional de TP8 con OpenCode

Según el resultado compartido, OpenCode leyó únicamente los scripts de TP8 y no modificó ni ejecutó SQL. Los hallazgos siguientes separan las observaciones del código de las comprobaciones que todavía requieren ejecutar SQLite. En el hallazgo 3, se trata como una ambigüedad del contrato entre catálogo y cliente móvil, no como una baja lógica demostrada.

Archivo	Hallazgo y decisión	Verificación / pendiente
`movil/schema_sqlite.sql` y `movil/semaforo.sql`	**Aceptado.** `cola_sync` permite más de una solicitud pendiente con UUID distintos para el mismo `id_local`. El semáforo evalúa ambas filas; esto duplica solicitudes/resultados de simulación, pero no ventas: el script solo actualiza estados locales y aclara que no es una venta.	Confirmado en una copia temporal: se insertaron dos UUID para un mismo borrador; ambas filas fueron aceptadas y el semáforo actualizó ambas a `RECHAZADO` según los datos de ese borrador. La base de entrega no se modificó. La regla de unicidad depende de la decisión de negocio.
`movil/cargar_cache.sql`	**Aceptado como riesgo de recuperación.** Si `snapshot_servidor` está vacío al ejecutar la carga de cache, el `UPDATE` marca todos los productos cacheados como no disponibles y eliminados. El comentario de `snapshot_sqlite.sql` indica conservar la instantánea anterior hasta tener un CSV nuevo, pero no hay una validación que impida ejecutar la carga con una instantánea vacía.	Confirmado en una copia temporal: el cache tenía 3 productos antes y después, y los 3 quedaron `eliminado=1` y `disponible=0` al cargar con snapshot vacío. La base de entrega no se modificó.
`servidor/snapshot_postgresql.sql`	**Aceptado como ambigüedad de contrato.** Un producto activo cuya categoría está inactiva sale con `disponible=false` y `eliminado=false`. Eso no representa una baja lógica del producto; puede ser correcto si el móvil oculta productos no disponibles. Conviene documentar cuál campo usa la interfaz para ocultar productos.	Verificado leyendo las expresiones de la vista. No se pudo ejecutar la prueba de exportación: PostgreSQL no respondió en `localhost:5432`. Queda pendiente probar una categoría inactiva con una conexión disponible.

Archivo	Hallazgo y decisión	Verificación / pendiente
`movil/semaforo.sql` y `movil/snapshot_sqlite.sql`	**Descartado.** No se encontró evidencia de que `CAST(s.stock AS INTEGER)` sea inseguro para el esquema previsto: `snapshot_servidor.stock` es `INTEGER NOT NULL` en una tabla `STRICT`, así que los valores inválidos no deberían quedar almacenados como texto.	Confirmado en la base original abierta en modo de solo lectura: `typeof(stock)` devolvió `integer` para todas las filas; `disponible` y `eliminado` solo contienen `t`/`f`.

La revisión inicial de OpenCode fue estática y limitada a TP8. Después se comprobaron las dos primeras observaciones en una copia temporal de SQLite y se verificaron los tipos y booleanos de la base original en modo de solo lectura. La prueba de la vista PostgreSQL queda pendiente porque no hubo conexión disponible en `localhost:5432`. No se modificaron los scripts ni la base original.

4. Defensa y estado

Para la defensa de 10 minutos: explicar el mapa de motores; mostrar `STRICT`, claves, checks y catálogo offline; ejecutar C2 o C4; explicar la prioridad del rechazo y la idempotencia; justificar la baja lógica.

Las pruebas SQLite del paquete verifican transacciones, C1, C7 y el fixture C2–C6. La base entregada fue comprobada con `PRAGMA integrity_check` (`ok`) y `PRAGMA foreign_key_check` (sin filas). En la copia de pruebas PostgreSQL se observaron los resultados C2–C5 y el control final de 5 pedidos/25 unidades de stock. C6 se comprobó localmente en SQLite. Kiro se utilizó para revisar y corregir los tres archivos de especificación. OpenCode aportó una revisión estática de TP8; las pruebas pendientes están identificadas como tales.